

Splines

Hannah Alter, Erin Nell, Gemma Rebollo Osma

May 2025

Contents

1	Introduction	1
1.1	Cubic Spline Interpolation	2
1.2	Not-a-Knot Boundary Conditions	2
2	Not-a-Knot Algorithm	3
3	Comparison with Natural and Clamped Boundary Conditions	
	Results and Discussion	4
3.1	Comparing Plots	4
3.2	Error Analysis	7
4	Comparing Built-In Spline Function	7
5	Exploration of Built-in Spline Functions in MATLAB and Discussion of Results	8
5.1	Comparing Plots	9
5.2	Error Analysis and Discussion	10
6	Conclusion	11
7	Appendix – MATLAB Code	12
7.1	not_a_knot_cubic_spline.m function	12
7.2	Implementing Built-in Functions and Plotting	13

1 Introduction

Cubic splines are a powerful tool in numerical analysis for interpolating data in a smooth, piecewise fashion. These splines are especially useful when the interpolating curve must appear visually smooth or when derivative continuity is important. These splines are used in many different contexts, like robotics, computer graphics, and data smoothing [5]. However, the choice of boundary conditions greatly influences the resulting spline behavior. In this project, we explore the Not-a-Knot boundary condition, which is less common in coursework

but widely used in professional libraries (such as MATLAB’s spline function). We will implement a custom version of this spline in MATLAB, test it against common alternatives (natural and clamped), and assess its performance on a variety of datasets. We also experiment with more advanced built-in MATLAB spline tools, such as `pchip`, `makima`, `csaps`, `spaps`, and the SLM Toolbox.

1.1 Cubic Spline Interpolation

Cubic spline interpolation constructs a smooth function $S(x)$ from a series of cubic polynomials $S_j(x)$ over sub-intervals $[x_j, x_{j+1}]$ that interpolate a given set of data points (x_i, y_i) . Each spline segment has the form:

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3 \quad (1)$$

The spline will pass through all given data points and is continuous, including continuous first and second derivatives across all internal knots. To determine the coefficients a_j, b_j, c_j, d_j , we solve a linear system that incorporates the constraints from interpolation and smoothness conditions, plus two extra equations from boundary conditions.

This process is formalized by the following definition provided by the Numerical Analysis textbook: Given a function f defined on $[a, b]$ and a set of nodes $a = x_0 < x_1 < \dots < x_n = b$, a cubic spline interpolant S for f is a function that satisfies the following conditions [1]:

- $S(x)$ is a cubic polynomial, denoted $S_j(x)$, on the subinterval $[x_j, x_{j+1}]$, for each $j = 0, 1, \dots, n - 1$.
- $S_j(x_j) = f(x_j)$ and $S_{j+1}(x_{j+1}) = f(x_{j+1})$, for each $j = 0, 1, \dots, n - 2$.
- $S_j(x_{j+1}) = S_{j+1}(x_{j+1})$, for each $j = 0, 1, \dots, n - 2$.
- $S'_j(x_{j+1}) = S'_{j+1}(x_{j+1})$, for each $j = 0, 1, \dots, n - 2$.
- $S''_j(x_{j+1}) = S''_{j+1}(x_{j+1})$, for each $j = 0, 1, \dots, n - 2$.

This definition ensures the spline interpolant is an accurate fit to the data and exhibits smooth behavior across the domain. Different choices of boundary conditions, such as natural, clamped, or not-a-knot, of which this paper will compare, influences how the spline behaves especially near the endpoints.

1.2 Not-a-Knot Boundary Conditions

The Not-a-Knot condition differs from natural (which forces the second derivative to be zero at the endpoints) and clamped (which forces known derivatives at the endpoints) [1]. For instance, the not-a-knot boundary conditions require third derivative continuity. Specifically, that the third derivatives of the first

two spline segments are equal at the second point (x_2) and that the third derivatives of the last two spline segments are equal at the second-to-last data point (x_{n-1}). The two conditions are as follows [6]:

$$S_1^{(3)}(x_2) = S_2^{(3)}(x_2)$$

$$S_{n-2}^{(3)}(x_{n-1}) = S_{n-1}^{(3)}(x_{n-1}).$$

This effectively merges the first two and last two spline segments into single cubics, hence the name “not-a-knot.” The spline remains C^2 -continuous, but with greater emphasis on internal uniformity than endpoint control.

One advantage of not-a-knot boundary conditions is that they result in greater smoothness. It results in more accurate approximations for the first and second derivatives of the function especially. Also, it does not require boundary derivative estimates, and it tends to be a frequent default setting in many software libraries, including MATLAB [2]. On the other hand, there are a few disadvantages to this boundary condition. Most notably, the accuracy of interpolation at the boundary interval edges tends to be low. To fix this, improved end conditions alter the parameter so that it is close to the boundary, which changes the spline knot locations and increases the accuracy of the interpolation [6]. Not-a-Knot boundary conditions can be used in several circumstances. General interpolation can incorporate this boundary condition, as well as with smooth curve functions without exact derivative control.

2 Not-a-Knot Algorithm

We implemented the not-a-knot cubic spline interpolation in a custom MATLAB function `not_a_knot_cubic_spline.m`, which is modeled after `natural_cubic_spline.m` and `clamped_cubic_spline.m` code provided. It computes the cubic spline coefficients a_i, b_i, c_i, d_i for each subinterval $[x_i, x_{i+1}]$. The algorithm proceeds as follows:

We begin with inputs of n data nodes $\{x_i\}$ and corresponding function values $\{y_i\}$, and compute interval lengths:

$$h_j = x_{j+1} - x_j \quad \text{for } j = 1, \dots, n-1.$$

To satisfy C^2 continuity and the Not-a-Knot boundary conditions, we construct a linear system $Ac = b$ to solve for the second derivatives $c_i = S''(x_i)$ at each node. The right-hand side vector b enforces continuity of first derivatives at internal nodes:

$$b_j = \frac{3}{h_j}(a_{j+1} - a_j) - \frac{3}{h_{j-1}}(a_j - a_{j-1})$$

The coefficient matrix A is tridiagonal for interior rows [2]:

$$A_{j,j-1} = h_{j-1}, \quad A_{j,j} = 2(h_{j-1} + h_j), \quad A_{j,j+1} = h_j$$

However, the Not-a-Knot boundary conditions modify the first and last rows to ensure third derivative continuity:

$$\begin{aligned} A_{1,1} &= h_2, & A_{1,2} &= -(h_1 + h_2), & A_{1,3} &= h_1 \\ A_{n,n} &= h_{n-2}, & A_{n,n-1} &= -(h_{n-1} + h_{n-2}), & A_{n,n-2} &= h_{n-1} \end{aligned}$$

Next, we solve for the second derivatives by solving for the linear system $Ac = b$. This gives us the second derivatives at the nodes, which ensures C^2 continuity. We then compute the remaining spline coefficients, b_j and d_j , since we now know a_j , c_j , and h_j :

$$b_j = \frac{a_{j+1} - a_j}{h_j} - \frac{h_j}{3}(2c_j + c_{j+1}), \quad d_j = \frac{c_{j+1} - c_j}{3h_j}$$

These equations are derived from taking derivatives of the cubic spline formula (1) and solving for the coefficients so that both $S'_j(x)$ and $S''_j(x)$ match at the endpoints x_j and x_{j+1} . The output are arrays of the coefficients a_j, b_j, c_j, d_j which define the cubic spline on each interval $[x_j, x_{j+1}]$.

To validate our implementation, we applied the spline to three test functions:

- $f(x) = x^3$: The spline reproduces the exact function, since it is a cubic polynomial.
- $f(x) = \sin(x)$: We compared the interpolated spline with the true function over dense query points and observed low interpolation error.
- $f(x) = |x - 5|$: This non-smooth function helps illustrate the spline's behavior at points where the first derivative is discontinuous.

See Appendix for full MATLAB code. We used six evenly spaced nodes in $[0, 10]$ and generated 200 query points in the same range for plotting interpolated curves. To better understand how these boundary conditions affect the behavior of the splines, we will compare with natural and clamped boundary conditions and plot.

3 Comparison with Natural and Clamped Boundary Conditions Results and Discussion

We now compare the behavior of the spline with the not-a-knot boundary conditions to the natural and clamped boundary conditions on the same datasets: $\sin(x)$, x^3 , and $|x - 5|$.

3.1 Comparing Plots

We will begin by analyzing Figure 1, which compares the three boundary conditions on $\sin(x)$. First, we note that all three methods pass through the data

points, as expected through interpolation. The clamped spline appears to provide the smoothest and most natural fit to the true sine wave, especially near the boundaries. This is most likely due to the use of derivatives in interpolation. The natural spline, in comparison, is similar to the clamped spline plot, but tends to flatten towards the ends. This is the result of the second derivatives being zero at the end points. Notably, the not-a-knot spline shows overshoot and deviation from the exact sine function near the endpoints. This is especially noticeable between the first two data-points and the last two data points. This is because the boundary conditions do not constrain the endpoints directly. Instead, they enforce an internal smoothness condition. This allows the spline to twist or bend more at the ends.

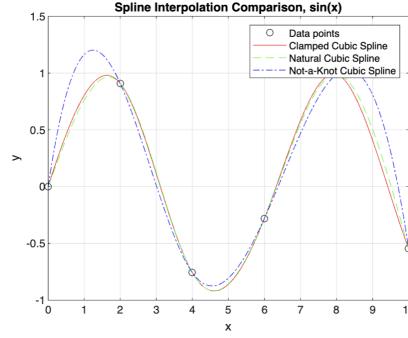


Figure 1: Plot of $f(x) = \sin(x)$ comparing results of the not-a-knot, clamped, and natural boundary conditions

The function x^3 is a smooth cubic polynomial, which aligns perfectly with the fundamental assumptions of cubic spline interpolation. In Figure 2, all three spline types nearly coincide with the true function and with each other across the entire domain. This result is expected since the true function is already a cubic. Thus, the interpolant reconstructed by any cubic spline method using evenly spaced data points will approximate it almost exactly. This also demonstrates that for smooth polynomials of degree three, the choice of boundary condition has minimal impact on the accuracy of the interpolant. However, the natural cubic spline very slightly deviates from the other plots. This is likely due to the nature of natural boundary conditions, which force the second derivative to be zero, when normally x^3 would have a non-zero second derivative. Thus, to meet these conditions, the spline must deviate from the true shape near the endpoints.

We can also see that not-a-knot performed much better on this dataset than for $\sin(x)$. As the not-a-knot boundary conditions make the first and last two segments smooth, as if they are a part of the same cubic curve, this works better for functions like x^3 , which are cubic everywhere. For functions like $\sin(x)$ which have lots of curvature changes, making the end segments smooth imposes behavior that does not match the functions true shape. When fewer

nodes are used (6 nodes, as seen in 1), this leads to a worse fit. This overall highlights the importance of matching boundary conditions to the context of the function to get the best results.

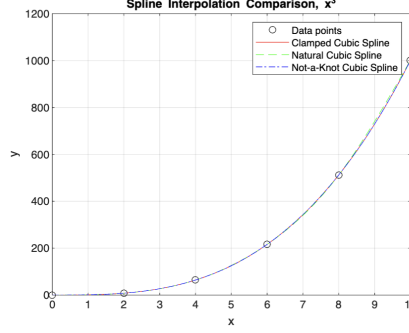


Figure 2: Plot of $f(x) = x^3$ comparing results of the not-a-knot, clamped, and natural boundary conditions

Looking at our plot for $f(x) = |x - 5|$, Figure 3, all three splines attempt to smooth out the non-differentiability at $x = 5$ since all the methods are continuous up to its second derivative [1].

First, the clamped spline enforces specific derivative values at the endpoints, which results in a slightly flatter curve near the boundaries. The natural spline, produces a more relaxed curve near the edges, though it still fails to preserve the original sharpness at $x = 5$. The not-a-knot spline slightly over-fits near the boundaries, as it merges the first and second intervals (and similarly the last two) into a single cubic segment. In this case, the not-a-knot spline appears slightly more flexible, but none of the methods can truly replicate the non-differentiable corner.

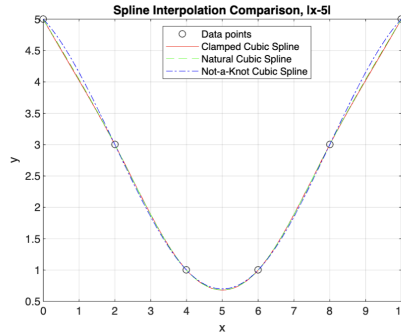


Figure 3: Plot of $f(x) = |x - 5|$ comparing results of the not-a-knot, clamped, and natural boundary conditions

3.2 Error Analysis

As seen in Table 1, we now compare the maximum interpolation errors across the three datasets which reflects what we observe in the plots:

For $\sin(x)$, all spline methods perform relatively well, with errors under 0.5. The clamped spline achieves the lowest error, due to its ability to incorporate known derivative information at the boundaries, which is beneficial for a smooth, periodic function like sine.

For x^3 , both the not-a-knot and clamped splines demonstrate high accuracy, with very small errors, while the natural spline performs significantly worse, exhibiting an error of approximately 11.78. This outcome is expected because cubic splines can exactly represent cubic polynomials if the boundary conditions are compatible; the natural spline, however, forces second derivatives to zero at the endpoints, a condition that conflicts with the curvature of x^3 , leading to poor approximation. Finally, for $|x - 5|$, a function that is not smooth at $x = 5$, the Not-a-Knot spline performs the best, with an error of approximately 0.695, whereas both Natural and Clamped splines have slightly larger errors. This is attributed to their stronger smoothness constraints, which struggle to approximate the corner. The not-a-knot spline, being more flexible at the boundaries, introduces less distortion near this singularity. Overall, this table illustrates how the choice of boundary condition significantly affects spline accuracy. Clamped splines are well-suited for smooth functions with known endpoint behavior, while natural splines may perform poorly when the boundary constraints contradict the function’s shape.

Boundary Condition	$\sin(x)$	x^3	$ x - 5 $
Not-a-Knot Max Error	4.259e-01	1.377e-13	6.950e-01
Natural Max Error	1.432e-01	1.178e+01	6.792e-01
Clamped Max Error	9.930e-02	1.137e-13	6.768e-01

Table 1: Maximum errors for different boundary conditions and datasets

4 Comparing Built-In Spline Function

We next wanted to compare MATLAB’s built in spline function to our own not-a-knot function. As seen in Figure 4, Figure 5, and Figure 6, the built-in spline function and our own function are the very similar, if not identical. This is because the built-in spline function also uses not-a-knot boundary conditions by default [4]. The similarity is also further confirmed by the computed maximum errors being almost identical, as seen in Table 1 and Table 2.

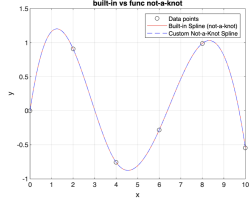


Figure 4: $f(x) = \sin(x)$

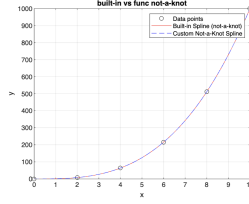


Figure 5: $f(x) = x^3$

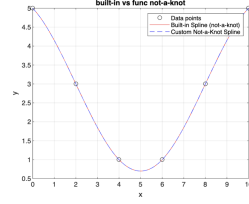


Figure 6: $f(x) = |x - 5|$

5 Exploration of Built-in Spline Functions in MATLAB and Discussion of Results

We then explored MATLAB's built-in spline options:

- `spline` (default: not-a-knot)
- `pchip` (C^1 , shape-preserving)
- `makima` (modified Akima, handles sharp transitions well)
- `csaps` (cubic smoothing spline)
- `spaps` (smoothing spline with error tolerance)
- `slmengine` (SLM Toolbox, supports geometric constraints)

The `spline` function in MATLAB implements a cubic spline with the Not-a-Knot boundary condition, ensuring C^2 continuity across all interpolated intervals. In contrast, `pchip` (Piecewise Cubic Hermite Interpolating Polynomial) is specifically designed to preserve the shape and monotonicity of the data. Unlike `spline`, `pchip` enforces only C^1 continuity, which helps to avoid overshoot and unwanted oscillations often seen in regions of high curvature. The `makima` method is constructed to reduce oscillations and handle sharp transitions. It also enforces C^1 continuity but introduces a smoothing effect without significantly compromising accuracy. The `csaps` function provides an approximation of the data rather than strict interpolation. It achieves this by minimizing a smoothness parameter that is defined by the user. `spaps` represents a generalization of `csaps`, where the degree of smoothness is governed by a specified error tolerance, allowing for a more precise balance between the smoothness of the resulting curve and its fidelity to the input data [4]. Finally, the `slmengine` from the Shape Language Modeling Toolbox offers a customizable spline tool, enabling users to define specific constraints such as monotonicity, curvature limits, or slope bounds [3].

5.1 Comparing Plots

Each method was applied to the same datasets, and plots were generated overlaying the curves for qualitative and visual comparison. These were also evaluated numerically by measuring maximum absolute error at the 200 query points.

To begin with, we explored the function $\sin(x)$, as seen in Figure 7, using built-in spline interpolation methods. The built in function `spline` produces a function identical to that of the `not_a_knot_cubic_spline.m` function. Similar to `not-a-knot`, `spline` is overshoot at the peak of the $\sin(x)$ waves and some deviation around the critical points. `Pchip` and `makima` also followed the general $\sin(x)$ shape, although they clearly appeared to be less smooth, in which `Makima` showed a slightly better approximation with a smoother tail than `pchip`. `csaps` seem most noticeably different, as it fails to pass through many of the data points. Because it aims to achieve a smoother overall curve than the other methods, it sacrifices some accuracy at the data points. Similarly, `spaps` produces a more accurate curve than `csaps` and is another smoothing spline. Finally, the `slmengine` curve appears to result in the closest approximation to the actual function, with some deviations. This likely results from this method being designed to preserve curvature trends [4], and thus it can be useful handling the behavior of sine.

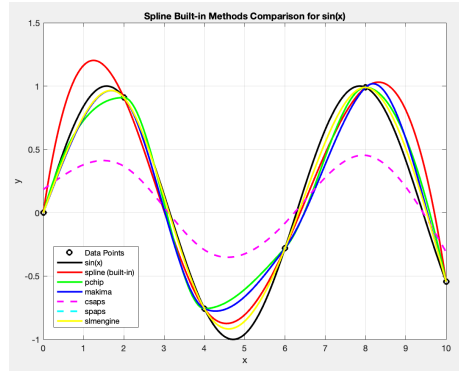


Figure 7: Plot of $f(x) = \sin(x)$ experimenting with built-in spline functions

Next, we compare the built-in interpolation methods for the x^3 dataset, Figure 8. All methods except for `csaps` appear to match the actual x^3 curve. `csaps` is very close to the actual curve; however, it deviates slightly from the data point at $x = 8$ and $x = 10$. Because x^3 is inherently smooth, this smoothing spline function prioritizes the smoothness, at the expense of ensuring accuracy.

Lastly, we compare the same methods for the function $|x - 5|$ using Figure 9. All of the methods clearly struggled to capture the sharp corner critical point at $x = 5$. Most noticeably, the graph using `pchip` produces a straight line in place of the vertex, with very little curvature. This demonstrates the goal of `pchip`, in which it is meant to preserve shape. Another noticeable line is `csaps`, which again misses some of the data points and rounds the corner

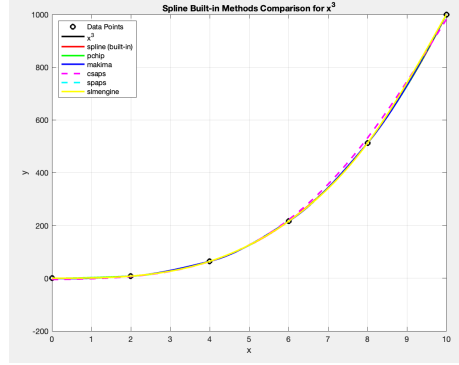


Figure 8: Plot of $f(x) = x^3$ experimenting with built-in spline functions

significantly. The other methods tend to better preserve the v-shape, but still have a rounding effect. Additionally, the built-in spline methods curves out towards the endpoints, as we saw with our own `not_a_knot_cubic_spline.m` function.

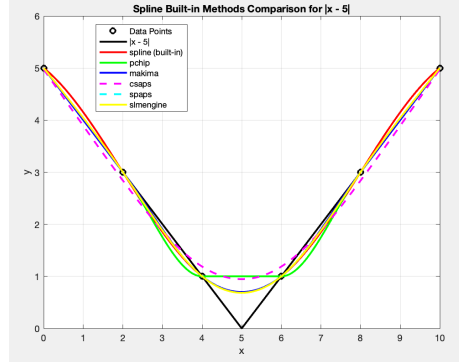


Figure 9: Plot of $f(x) = |x - 5|$ experimenting with built-in spline functions

5.2 Error Analysis and Discussion

The maximum interpolation errors observed in the three distinct datasets are presented in Table 2, and these numerical results corroborate the visual trends identified in the previous plots.

In the case of $\sin(x)$, we observe that `spaps` and `slmengine` yield the smallest maximum errors, indicating their accuracy for this function. In the mid-range of performance are `makima` and `pchip`, which show moderate errors. While these methods generally follow the sine wave shape well, they show slightly more deviations, particularly near the endpoints. Conversely, `spline` and `csaps` have the highest error. This suggests that `csaps`'s inherent smoothing behavior may

be detrimental when attempting to match the sharper curvature of the sine wave.

Looking at the cubic function x^3 , the built-in `spline` method performs exceptionally well, achieving essentially exact interpolation at the given data points. In contrast, `spaps`, `slmengine`, and `csaps` all demonstrate significantly larger errors. As these are smoothing splines and are not optimized for perfect reproduction of polynomials, this is an unsurprising outcome. Interestingly, `pchip` and `makima` fall in an intermediate range. Despite appearing visually accurate, they still introduce small but measurable deviations.

Finally, we analyze the results for $|x-5|$. `spaps` and `slmengine` again provide the best results with the smallest errors. This suggests their effectiveness in capturing the overall trend despite the function’s non-smooth point. `spline` and `makima` perform slightly worse but remain relatively close in accuracy. However, `pchip` and `csaps` show the poorest performance. `pchip` flattens the vertex at the corner, resulting in a larger deviation from the true function. Furthermore, the smoothing effect of `csaps` causes a similar loss in accuracy at this sharp point.

Interpolation Method	$\sin(x)$	x^3	$ x - 5 $
Spline	4.259e-01	1.137e-13	6.950e-01
pchip	3.542e-01	1.557e+00	9.950e-01
makima	2.965e-01	4.363e+00	6.950e-01
csaps	6.526e-01	2.081e+01	9.413e-01
spaps	1.037e-01	1.178e+01	6.797e-01
slmengine	1.033e-01	1.178e+01	6.792e-01

Table 2: Maximum errors for different interpolation methods and functions

6 Conclusion

Our results show that the not-a-knot spline is an effective general-purpose interpolant. Its performance is very close to MATLAB’s built-in spline function and works well when derivative data is not available. In scenarios requiring endpoint accuracy, clamped splines are superior. When the goal is to preserve shape or avoid overshoots, piecewise Hermite interpolants (like `pchip` and `makima`) offer better behavior. Smoothing splines (`csaps`, `spaps`) allow for approximation rather than interpolation, which is helpful for noisy data. The SLM Toolbox is highly customizable, but is best suited for advanced tasks that require constraint control.

In terms of our error analysis, these results confirm that the not-a-knot boundary conditions allows a performance very closely to clamped on smooth data without known derivatives, although a clamped spline is best when the derivative information is available. For non-smooth data, shape-preserving in-

terpolants like pchip outperform smoother cubic splines. Our analysis and discussion highlights the importance of choosing which spline function to use based off of the context of the function and the goal of what would like to be accomplished.

Throughout this paper, we successfully implemented a not-a-knot spline algorithm from scratch, verified its accuracy against MATLAB's built-in functions, and compared its performance across different types of functions and spline tools. The not-a-knot condition proves robust and smooth, though slightly less intuitive than clamped or natural approaches. In the future, we would like to test these functions on real-world noisy data and compare computational efficiencies.

7 Appendix – MATLAB Code

7.1 not_a_knot_cubic_spline.m function

```
function [a,b,c,d] = not_a_knot_cubic_spline(x, y)
% Implements cubic spline interpolation
% with not-a-knot boundary conditions
n = length(x);
a = y(:);
h = diff(x);

% Set up right-hand side (b_RHS)
b_RHS = zeros(n,1);
for j = 2:n-1
    b_RHS(j) = 3/h(j)*(a(j+1) - a(j)) - 3/h(j-1)*(a(j) - a(j-1));
end

% Set up the tridiagonal matrix A
main_diag = zeros(n,1);
upper_diag = zeros(n-1,1);
lower_diag = zeros(n-1,1);

for j = 2:n-1
    main_diag(j) = 2*(h(j-1) + h(j));
    upper_diag(j) = h(j);
    lower_diag(j-1) = h(j-1);
end

% Not-a-knot conditions at first and last row
% First row (corresponds to x_1 = not-a-knot at x_2)
A = zeros(n);
A(1,1) = h(2);
A(1,2) = -(h(1)+h(2));
```

```

A(1,3) = h(1);

% Last row (not-a-knot at  $x_{n-1}$ )
A(n,n) = h(n-2);
A(n,n-1) = -(h(n-1)+h(n-2));
A(n,n-2) = h(n-1);

% Fill the rest of A
for j = 2:n-1
    A(j,j-1) = h(j-1);
    A(j,j) = 2*(h(j-1)+h(j));
    A(j,j+1) = h(j);
end

% Solve the linear system
c = A\b_RHS;

% Compute b and d coefficients
b = zeros(n-1,1); d = zeros(n-1,1);
for j = 1:n-1
    b(j) = 1/h(j)*(a(j+1) - a(j)) - h(j)/3*(2*c(j) + c(j+1));
    d(j) = (c(j+1) - c(j)) / (3*h(j));
end

% Trim a and c to match b and d
a = a(1:end-1);
c = c(1:end-1);

```

7.2 Implementing Built-in Functions and Plotting

```

% Dataset
x = linspace(0, 10, 6);
y = x.^3; % or: sin(x), abs(x-5)

xq = linspace(min(x), max(x), 1000);

% Evaluate built-in splines
spline_builtin = spline(x, y, xq);
pchip_builtin = pchip(x, y, xq);
makima_builtin = makima(x, y, xq);

% csaps
csaps_fn = csaps(x, y);
csaps_builtin = fnval(csaps_fn, xq);

```

```
% spaps
[spaps_fn , ~] = spaps(x, y, 1e-6);
spaps_builtin = fnval(spaps_fn , xq);

% slmengine
slm = slmengine(x, y, 'plot ', 'off ');
slm_builtin = slmeval(xq, slm);
```

References

- [1] Richard L. Burden, J. Douglas Faires, and Annette M. Burden. *Numerical Analysis*. CENGAGE Learning, 10th edition, 2015.
- [2] J. Cooper. *Cubic Splines*. University of Maryland.
- [3] J. D’Errico. Slm - shape language modeling [matlab central file exchange], 2023.
- [4] MathWorks. Spline, matlab documentation, 2023.
- [5] Jens Ogniewski. Spline interpolation in real-time applications using three control points. 01 2019.
- [6] Meng Sun, Lin Lan, Chun-Gang Zhu, and Fengchun Lei. Cubic spline interpolation with optimal end conditions. *Journal of Computational and Applied Mathematics*, 425:115039, 2023.